

Laboratorio 7 — NLP end-to-end y Embeddings

CC3092 · Deep Learning y Sistemas Inteligentes · WikiText-103, AG News y GloVe 100d

1. Investigación: embeddings en PyTorch y gensim

`nn.Embedding(num_embeddings, embedding_dim, padding_idx, sparse)` es una tabla $|V| \times d$: el ID del token selecciona una fila (equivalente a one-hot $\times$ matriz, pero indexando). `padding_idx` fija una fila que nunca recibe gradiente (lo verificamos en el notebook) y `sparse=True` produce gradientes solo para las filas usadas, lo que permite `SparseAdam` con vocabularios grandes. `nn.Embedding.from_pretrained(weights, freeze)` carga vectores ya entrenados: `freeze=True` los deja fijos y `False` permite fine-tuning. `nn.EmbeddingBag(mode)` busca y agrega en un paso los vectores de cada documento (mean, sum o max) a partir de un tensor 1D concatenado y de `offsets`, que marcan dónde empieza cada documento; evita el padding. En `gensim`, `Word2Vec` (`sg`, `vector_size`, `window`, `negative`, `sample`, `min_count`, `epochs`) entrena en C multihilo y `KeyedVectors` consulta (`most_similar`, `evaluate_word_pairs`).

CBOW predice la palabra central desde el promedio del contexto (rápido, favorece palabras frecuentes); **skip-gram** predice cada palabra del contexto desde la central (más pares por token, mejor con palabras raras). `Word2Vec` usa **dos matrices**: W (palabra como central) y W' (palabra como contexto). Así una palabra no necesita ser similar a sí misma, y se conserva W . **Word2Vec es predictivo** (SGD sobre ventanas locales); **GloVe es de conteo**: ajusta $w_i \cdot \tilde{w}_j + b_i + \tilde{b}_j \approx \log X_{ij}$ sobre la matriz global de co-ocurrencias, con pesos $f(X_{ij})$.

2. Datos y decisiones del pipeline

WikiText-103 (train): 29 444 artículos, 1 801 350 líneas y 101,4 M tokens crudos (85,9 M tras normalizar). Entrenamos con los primeros 8 802 artículos: **25,0 M tokens** y 251 800 líneas. AG News: 108 000 / 12 000 (validación estratificada al 10 %) / 7 600 (test oficial, usado una sola vez por inicialización).

Decisión	Elección y motivo
Normalización	Minúsculas (GloVe 6B está en minúsculas); se quitan títulos = ... = y la puntuación; @, @/., @ reconstruyen 20,000 y 3.5 (GloVe guarda números); @-@ separa compuestos; contracciones estilo Treebank (is + n't, army + 's), igual que el tokenizador de Stanford usado por GloVe. Sin tokens inventados (<num>), que no existen en GloVe y bajarían la cobertura.
Tokenizador	Regex propia vs NLTK Treebank en 12 oraciones: coinciden en contracciones (do/n't); difieren en guiones (well-known unido en NLTK), abreviaturas (dr. vs dr), alfanuméricos (1990s, q3 se parten en los nuestros), puntuación y símbolos (\$, %, URLs, que NLTK conserva y nosotros descartamos).
Zipf	Recta en log-log con pendiente $-1,06$ (≈ -1): sí se cumple. Las top 10 / 1 000 / 30 000 palabras cubren el 24,3 / 64,5 / 95,8 % de los tokens.
Vocabulario	<code>min_count</code> 1 / 5 / 10 / 20 $\rightarrow$ 271 737 / 90 566 / 61 449 / 41 805 palabras y 0 / 1,18 / 1,94 / 3,01 % de tokens desconocidos. Elegimos 5.
Subsampling	<code>keep</code> = $(\sqrt{f/t}+1) \cdot t/f$ con $t = 10^{-4}$: se descarta el 96,2 % de «the», el 94,1 % de «of» y el 93,8 % de «and» (medido: 96,2 / 94,2 / 93,7 %); «king» pierde el 25 % y «computer», el 0 %. Ayuda porque esos pares no aportan información, acelera el entrenamiento y amplía la ventana efectiva.
Pares por epoch	Ventana 5 sin cruzar líneas: 239,6 M pares (24,7 M tokens) $\rightarrow$ 141,3 M con subsampling (14,9 M tokens); con la ventana dinámica de word2vec, 85,7 M.

3. Iteraciones de skip-gram con negative sampling (PyTorch)

Implementación propia: dos `nn.Embedding(sparse=True)` (entrada inicializada en $\pm 0,5/d$ y salida en cero), pérdida $-\log \sigma(u_o \cdot v_c) - \sum \log \sigma(-u_k \cdot v_c)$, negativos de unigrama^{0,75}, pares generados en GPU en cada epoch (subsampling nuevo, ventana dinámica), `SparseAdam` con lr lineal decreciente y lotes de 8 192 pares. Se evalúa al final de cada epoch (analogías en el top 30 000 y WordSim-353); SimLex queda reservado. GPU: RTX 4050 Laptop (6 GB). Base: $d = 100$, ventana 5, 5 negativos, $t = 10^{-4}$, `min_count` 5, lr $2 \cdot 10^{-3}$, 5 epochs, 24,7 M tokens.

ID	Cambio respecto a la base	Tokens	Vocab.	Pérdida	Sem.	Sint.	Total	WS-353 p	s/epoch	GPU MB
I01	base	24,7 M	90 566	2,19	27,6	43,5	38,8	0,648	31	1 546
I02	$d = 50$	24,7 M	90 566	2,24	19,4	32,6	28,7	0,613	29	1 431
I03	$d = 300$	24,7 M	90 566	2,11	39,1	48,1	45,4	0,681	150	2 005
I04	ventana 2	24,7 M	90 566	2,09	22,0	40,5	35,0	0,600	23	1 306
I05	ventana 10	24,7 M	90 566	2,25	32,5	43,6	40,3	0,674	52	1 926
I06	15 negativos	24,7 M	90 566	3,12	32,3	43,0	39,8	0,653	63	1 578
I07	$t = 10^{-3}$	24,7 M	90 566	2,23	27,3	43,5	38,7	0,649	37	1 615
I08	<code>min_count</code> 20	24,2 M	41 805	2,24	28,3	42,9	38,6	0,658	26	1 416
I09	lr $5 \cdot 10^{-3}$	24,7 M	90 566	2,16	36,4	45,1	42,5	0,661	29	1 547
I10	corpus 25 %	6,1 M	43 162	2,24	9,2	15,5	13,6	0,545	6	961
I11	corpus 50 %	12,3 M	62 918	2,21	18,3	29,3	25,8	0,617	14	1 165
I12	ventana 10 + 15 neg. + lr $5 \cdot 10^{-3}$, 8 epochs	24,7 M	90 566	3,15	47,2	46,3	46,6	0,665	154	1 959

Analogías en %, con 3CosAdd. Mejor modelo de $d = 100$ (para compararlo con GloVe-100): **I12**, con 1 234 s de entrenamiento en total. **¿Una pérdida más baja implica mejores embeddings? No:** I04 tiene la pérdida más baja (2,09) y analogías peores que la base; I12 tiene la más alta (3,15) y es el mejor. La pérdida mide una tarea que cambia con la ventana y el número de negativos, no la calidad del espacio. Los vecinos de control son coherentes desde el epoch 1 (*france* → *italy*, *spain*; *january* → *february*, *december*) y se afinan con los epochs.

4. Verificación de la aritmética vectorial

`analogia(a, b, c, k)` (3CosAdd sobre vectores normalizados, excluyendo a, b y c) devuelve el mismo top-5 que `most_similar` de gensim en las 18 analogías y los tres modelos. También verificamos nuestro 3CosMul contra `most_similar_cosmul`: 66,25 % en ambos sobre 400 preguntas.

Tipo (3 analogías c/u)	Ejemplo (SGNS)	SGNS	gensim	GloVe
Género	man:woman :: king: queen (0,715), rango 1	3/3	1/3	3/3
País-capital	france:paris :: italy:? → vienna; rome en el rango 4 (0,645)	1/3	2/3	2/3
País-gentilicio	france:french :: germany: german (0,868)	3/3	3/3	3/3
Comparativo/superlativo	good:better :: bad: worse ; strongest en el rango 2	2/3	1/3	2/3
Tiempo verbal	walk:walked :: run: ran ; go:went :: see:? → came	2/3	3/3	2/3
Plural	car:cars :: child: children (0,721)	3/3	3/3	3/3
Total · 1.º puesto sin excluir a, b, c (palabra de la consulta)		14/18 · 14/18	13/18 · 15/18	15/18 · 7/18

Sin excluir las palabras de la consulta, la primera suele ser c o b (king, sister, prince, run...): $b - a + c$ sigue más cerca de c que de la respuesta, cuyo coseno con el vector resultado queda entre 0,55 y 0,87 en SGNS. **Evaluación sistemática** con el vocabulario compartido (las 30 000 palabras más frecuentes de WikiText presentes en los tres modelos): cobertura de 11 892 / 19 544 preguntas (60,8 %), las mismas para los tres.

Categoría (3CosAdd, %)	SGNS	gensim	GloVe	Paralelismo SGNS	Paralelismo GloVe
capital-common-countries	69	68	94	0,30	0,58
capital-world	59	58	89	0,23	0,57
currency	0	0	5	0,21	0,27
city-in-state	36	33	31	0,17	0,34
family	48	45	88	0,23	0,51
adjective-to-adverb	14	11	28	0,09	0,27
opposite	10	11	27	0,11	0,23
comparative / superlative	45 / 25	42 / 21	80 / 61	0,25 / 0,24	0,42 / 0,47
present-participle / past-tense	39 / 61	36 / 54	72 / 59	0,10 / 0,17	0,32 / 0,29
nationality-adjective	84	85	97	0,41	0,75
plural / plural-verbs	51 / 28	46 / 29	79 / 58	0,15 / 0,20	0,32 / 0,36
Semánticas (add / mul)	47,2 / 43,9	45,3 / 41,5	61,2 / 58,7	0,20	0,41
Sintácticas (add / mul)	46,4 / 42,6	43,5 / 39,8	68,3 / 66,9		
Total (add / mul)	46,7 / 43,0	44,0 / 40,3	66,2 / 64,5		

El desglose por categoría con 3CosMul está en el notebook; 3CosMul quedó unos 3 puntos por debajo de 3CosAdd en los tres modelos. **Paralelismo** (sección 5.3): coseno promedio entre los vectores diferencia $b - a$ de los pares de una misma relación; 1 indicaría flechas paralelas. t-SNE de 457 palabras en 9 grupos con el mejor SGNS: se forman los grupos esperados (países, números, meses y días, deportes, comida, animales, profesiones, colores y verbos), con silhouette de 0,49 en 2D y 0,19 en el espacio original (coseno). Solo algunos colores metálicos (silver, bronze, golden) caen junto a los animales.

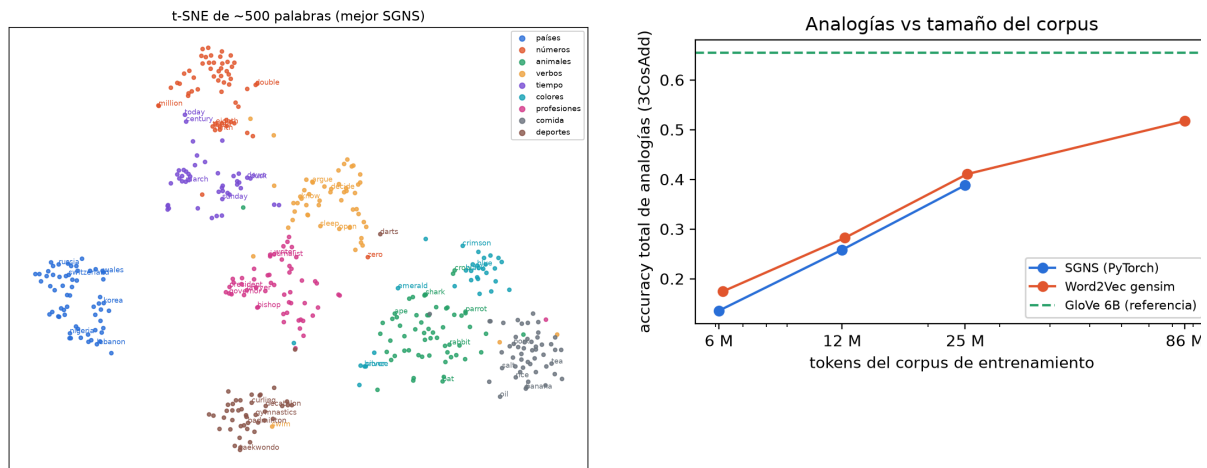


Figura 1. Izquierda: t-SNE del mejor SGNS. Derecha: analogías vs tokens del corpus (configuración base; GloVe evaluado en su propio top 30 000).

5. Clasificación de AG News (texto → tokens → vectores → EmbeddingBag(mean) → MLP 100-128-4)

Mismo normalize y tokenizador de la sección 2. Vocabulario del clasificador: 43 299 palabras de train con frecuencia ≥ 2 (las filas ausentes en el embedding se inicializan al azar). Tokens de AG News fuera del vocabulario: **SGNS 3,0 %**, gensim 3,0 % y **GloVe 0,7 %**. Adam con lr 10^{-3} , lotes de 256, ≤ 10 epochs y early stopping por F1 de validación. Baseline: TF-IDF de uni- y bigramas (422 568 rasgos) + regresión logística.

Modelo (validación)	Acc.	Prec.	Recall	F1	Parámetros	Entrenables	Tiempo	F1 test
TF-IDF + LR	92,64	92,62	92,64	92,62	1 690 276	1 690 276	41 s	92,13
Aleatorio	91,35	91,35	91,35	91,34	4 343 344	4 343 344	8,6 s	90,69
SGNS congelado	88,26	88,30	88,26	88,26	4 343 344	13 444	3,7 s	–
SGNS fine-tuning	92,44	92,44	92,44	92,43	4 343 344	4 343 344	4,7 s	91,72
gensim congelado / FT	87,92 / 92,44	87,98 / 92,44	87,93 / 92,44	87,92 / 92,43	4 343 344	13 444 / 4,3 M	2,6 / 4,9 s	– / 91,62
GloVe congelado	90,29	90,30	90,29	90,29	4 343 344	13 444	3,7 s	–
GloVe fine-tuning	92,47	92,48	92,47	92,46	4 343 344	4 343 344	6,8 s	91,73

La mejor variante de cada inicialización (fine-tuning en los tres casos) se evaluó una sola vez en test. En las matrices de confusión el error dominante es Business ↔ Sci/Tech (noticias de empresas tecnológicas); Sports es casi perfecta. Spearman en SimLex-999: SGNS 0,279, gensim 0,290 y GloVe 0,298, bajos en los tres porque los embeddings por co-ocurrencia capturan *relación*, no *similitud* estricta (los antónimos quedan cerca).

6. Comparación de los tres conjuntos de embeddings

	SGNS propio (I12)	Word2Vec gensim	GloVe 6B
Tokens del corpus · vocabulario · dimensión	24,7 M · 90 566 · 100	25,0 M · 90 566 · 100	6 000 M · 400 000 · 100
Tiempo y hardware	1 234 s (8 epochs), RTX 4050 Laptop, 2,0 GB de pico	734 s (8 epochs), CPU de 16 hilos	~85 min para X (1 hilo) + 50 iteraciones; Xeon E5-2658 dual a 2,1 GHz, 32 núcleos
Analogías 3CosAdd: sem. / sint. / total	47,2 / 46,4 / 46,7	45,3 / 43,5 / 44,0	61,2 / 68,3 / 66,2
Analogías 3CosMul: sem. / sint. / total	43,9 / 42,6 / 43,0	41,5 / 39,8 / 40,3	58,7 / 66,9 / 64,5
Spearman WordSim-353 · SimLex-999	0,665 · 0,279	0,660 · 0,290	0,533 · 0,298
Coseno promedio entre vectores diferencia	0,204	0,198	0,406
OOV en AG News · F1 de test	3,0 % · 91,72	3,0 % · 91,62	0,7 % · 91,73

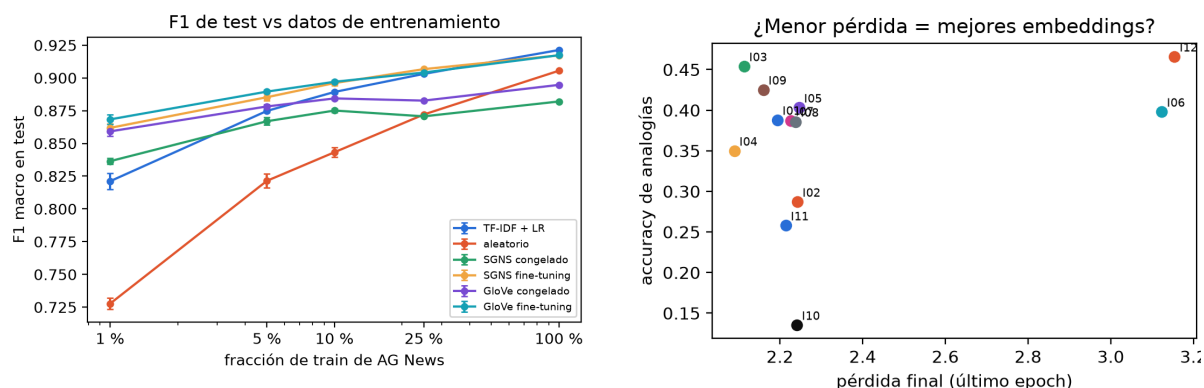


Figura 2. Izquierda: F1 de test vs fracción de train de AG News (media $\pm$ d.e. de 3 semillas hasta el 10 %). Derecha: pérdida final vs analogías por iteración.

7. Discusión

- **¿Se cumple la aritmética vectorial?** Parcialmente: el 46,7 % de las analogías y 14 de 18 propias. Funciona mejor en gentilicios (84 %), capitales (59-69 %) y pasado verbal (61 %), y falla en monedas (0 %), opuestos (10 %) y adjetivo $\rightarrow$ adverbio (14 %): relaciones raras en el corpus o que no son un desplazamiento constante (sus flechas casi no son paralelas: coseno de 0,09 a 0,11). En nuestro SGNS lo semántico y lo sintáctico quedan parejos (47,2 frente a 46,4 %); GloVe es más fuerte en lo sintáctico.
- **¿El « $\approx$ » es una igualdad?** No. Sin excluir la consulta, el vector más cercano a king - man + woman es king, y esto pasa en 14 de 18 analogías: el desplazamiento woman - man es pequeño frente a la norma de king. El coseno con la respuesta correcta nunca llega a 1 (de 0,55 a 0,87), y las diferencias de una misma relación tienen un coseno medio de solo 0,20 (0,41 en GloVe). La analogía funciona como búsqueda del vecino más cercano en un espacio ruidoso, no como una igualdad algebraica.
- **¿Qué tan lejos de GloVe y por qué?** 19,5 puntos (46,7 frente a 66,2 %). Con la misma configuración, gensim pasa de 17,4 % (6 M tokens) a 28,2 % (12 M), 41,1 % (25 M) y 51,7 % (86 M): entre 11 y 13 puntos por cada duplicación del corpus, y la tendencia sigue al alza. GloVe usa 240 veces más texto. La mayor parte de la distancia se explica por el corpus; los hiperparámetros, con el corpus fijo, aportaron +7,8 puntos (de 38,8 a 46,6 %), y el método por sí mismo poco: SGNS y GloVe empatan en F1 de AG News y SGNS supera a GloVe en WordSim.
- **¿Nuestro SGNS se parece a gensim?** Sí: 38,8 frente a 41,1 % con la base y 46,7 frente a 44,0 % con la mejor configuración, WordSim 0,665 frente a 0,660, la misma curva con el tamaño del corpus y las mismas categorías fuertes y débiles. Las diferencias se deben al optimizador (SparseAdam con lotes de 8 192 pares frente a SGD por par con lr lineal de 0,025, por lo que el lr no es trasladable), a la tabla de negativos y a la aleatoriedad. Nuestro SGNS en GPU (31 s/epoch) no supera a gensim en CPU (26 s/epoch), cuyo código en C es muy eficiente.
- **Dimensión, ventana y negativos.** $d = 50 \rightarrow 100 \rightarrow 300$ da 28,7 $\rightarrow$ 38,8 $\rightarrow$ 45,4 %, con 5 veces más tiempo. La ventana 2 $\rightarrow$ 5 $\rightarrow$ 10 sube las semánticas de 22,0 a 27,6 y a 32,5 %, mientras las sintácticas casi no cambian (40,5 / 43,5 / 43,6 %): **las ventanas pequeñas favorecen relativamente lo sintáctico** (la morfología está en los vecinos inmediatos) y las grandes aportan el tema (lo semántico). 15 negativos dan +1 punto con el doble de tiempo; el lr fue el ajuste más rentable (+3,7 puntos).
- **AG News.** Con todos los datos, los preentrenados con fine-tuning (91,7 %) superan a los aleatorios (90,7 %), pero no a TF-IDF con bigramas (92,1 %): con 108 000 noticias las palabras clave bastan. Congelar rindió menos en todos los casos (88,3 frente a 92,4 % en validación con SGNS), aunque entrena 320 veces menos parámetros. **Con el 1 % de los datos** (1 080 noticias) el orden se invierte: GloVe FT 86,8 %, SGNS FT 86,2 %, GloVe congelado 85,9 %, SGNS congelado 83,6 %, TF-IDF 82,1 % y aleatorio 72,8 %. Los embeddings aportan el conocimiento que faltan los datos (+14 puntos frente a aleatorio) y congelar queda a menos de 1 punto del fine-tuning en GloVe. Conviene congelar cuando hay pocos datos o poco cómputo, y hacer fine-tuning cuando sobran datos.

8. Conclusiones

- Un SGNS escrito desde cero en PyTorch reproduce a gensim (incluso lo supera ajustado: 46,7 frente a 44,0 %), lo que valida la implementación: pares con subsampling y ventana dinámica, negativos de unigrama^{0,75} y dos matrices.
- El tamaño del corpus domina la calidad: $\times 4$ tokens equivale a +25 puntos en analogías, más que cualquier hiperparámetro. Con 25 M tokens no se alcanza a GloVe (6 000 M), aunque sí se le gana en WordSim.
- La aritmética vectorial es aproximada: funciona por vecino más cercano y excluyendo la consulta, con flechas lejos de ser paralelas.
- La pérdida de entrenamiento no sirve para elegir embeddings; hay que evaluarlos en tareas intrínsecas (analogías, similitud) y extrínsecas (clasificación).
- En una tarea real, los embeddings preentrenados valen sobre todo cuando hay pocos datos etiquetados; con muchos datos, un TF-IDF bien hecho sigue siendo un rival muy difícil.

Referencias: Mikolov et al. (2013), *Distributed Representations of Words and Phrases and their Compositionality*, NeurIPS. Pennington, Socher y Manning (2014), *GloVe: Global Vectors for Word Representation*, EMNLP. Levy y Goldberg (2014), *Linguistic Regularities in Sparse and Explicit Word Representations*, CoNLL (3CosMul). Merity et al. (2016), *Pointer Sentinel Mixture Models* (WikiText-103).

Repositorio: <https://github.com/ecarcamo/CC3092-DEEP-LEARNING/tree/main/lab7> — vectores del mejor modelo en `results/vectors/best_sgns.kv`